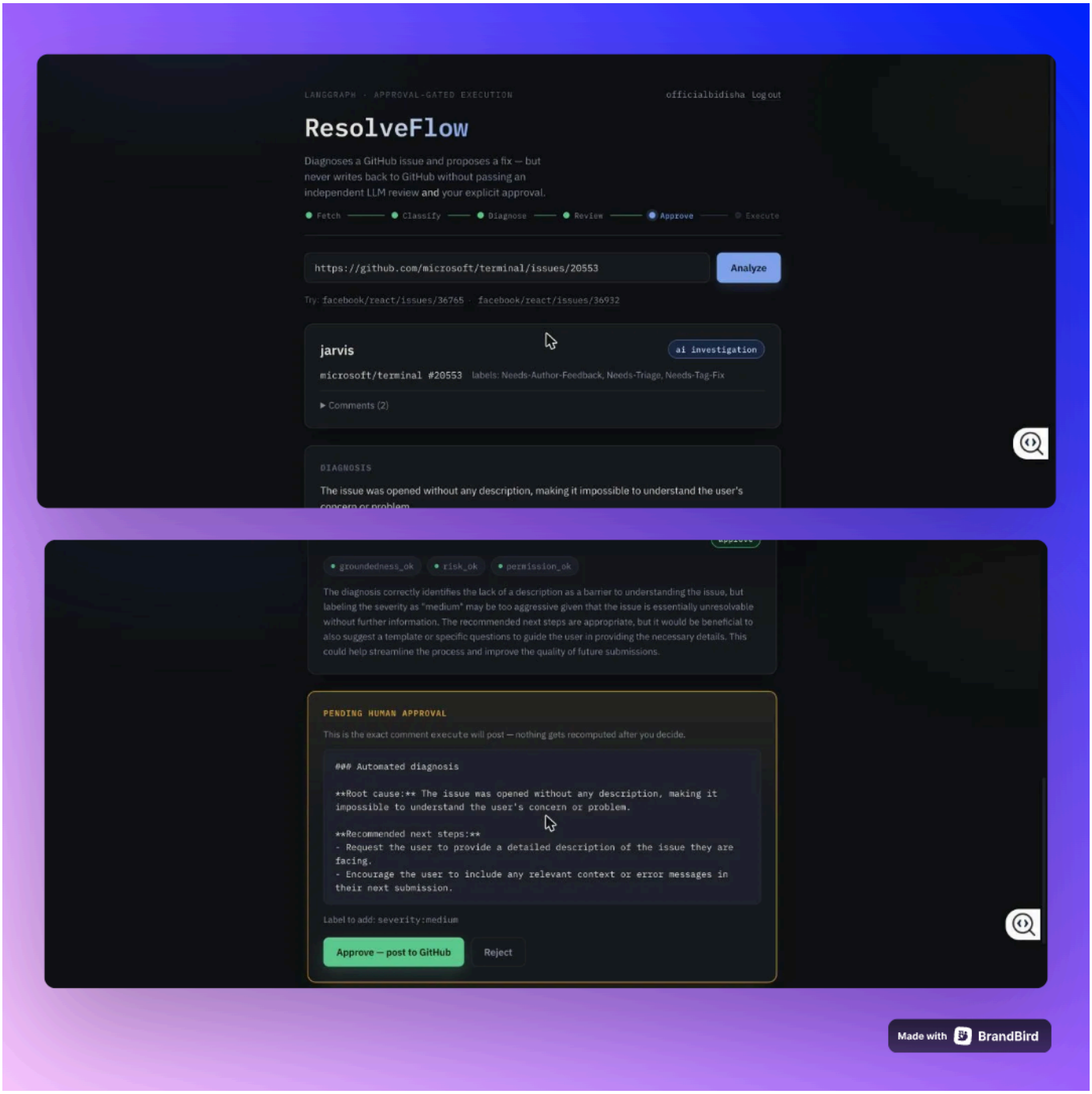
Appendix B — ResolveFlow design docs

Source: `README.md`

ResolveFlow



A LangGraph agent that diagnoses GitHub issues and proposes fixes — but never touches GitHub without passing an independent LLM review *and* an explicit human approval. [![Python](https://img.shields.io/badge/Python-3.12-3776AB?style=flat-square&logo=python&logoColor=white)](pyproject.toml) [![LangGraph](https://img.shields.io/badge/LangGraph-StateGraph-1C1C1C?style=flat-square)](graph/build.py) [![FastAPI](https://img.shields.io/badge/FastAPI-async-009688?style=flat-square&logo=fastapi&logoColor=white)](app/main.py) [![React](https://img.shields.io/badge/React-TypeScript-61DAFB?style=flat-square&logo=react&logoColor=black)](frontend) [![Postgres](https://img.shields.io/badge/Postgres-checkpointed-4169E1?style=flat-square&logo=postgresql&logoColor=white)](docs/CHECKPOINTING.md) [![Pinecone]

(https://img.shields.io/badge/Pinecone-RAG-000000?style=flat-square)](tools/retrieval.py) [![OpenAI](https://img.shields.io/badge/OpenAI-LLM-412991?style=flat-square&logo=openai&logoColor=white)](graph/nodes) ![status](https://img.shields.io/badge/status-approval--gated%20execution%20live-brightgreen?style=flat-square)

Live: resolveflow-web-officialbidishas-projects.vercel.app · API: [resolveflow-1h99.onrender.com](https://resolveflow-1h99.onrender.com/api/health)

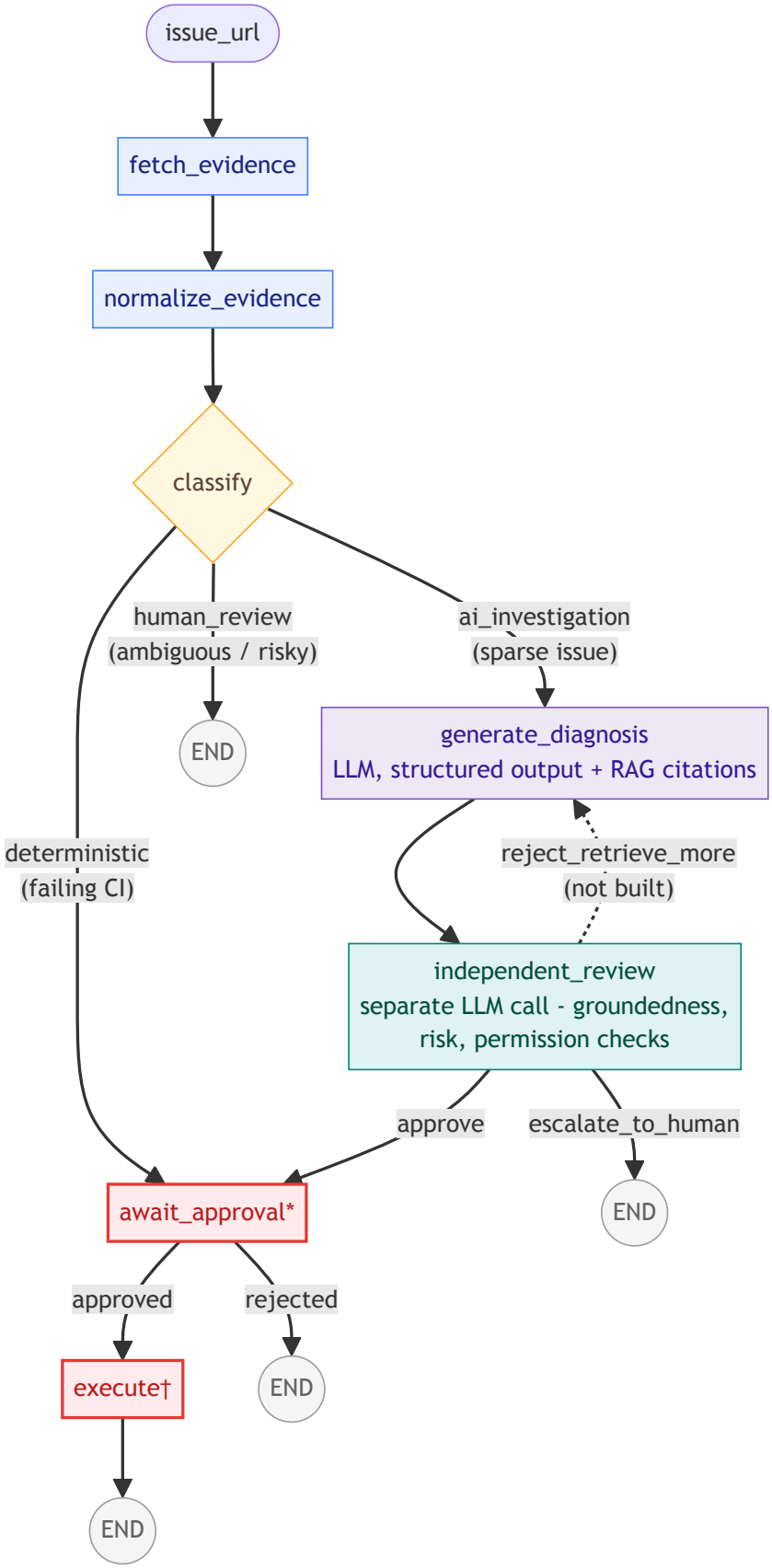
ResolveFlow takes a GitHub issue URL, gathers evidence, classifies the issue, and — depending on that classification — either runs a deterministic action, kicks off an LLM investigation with cited retrieval, or escalates straight to a human. The core architectural bet: **reasoning and execution are separated by construction, not convention.** A second, independent LLM call reviews the first model's diagnosis before anything reaches a human for approval, and the single node allowed to write to GitHub refuses to run without an explicit `approved` flag — checked in the node itself, not just at an API boundary.

Contents: Why this exists · Architecture · Core concepts · Status · Setup · Evaluation · Running · Roadmap

Why this exists

This is a portfolio project, built to show a real, working slice of agentic system design: state machines over prompt chains, a hard boundary between the part of the system that reasons and the part that acts, and an evidence/citation trail a reviewer can actually audit. It is not trying to be production-hardened — see Status for exactly what is real, what is a deliberate placeholder, and what is simply unbuilt.

Architecture



red = the human-approval gate and the only node with side effects · purple = LLM reasoning · teal = the independent LLM reviewer · amber = deterministic routing · blue = evidence gathering

`*` `await_approval` builds the exact comment/label that would be posted, then pauses via LangGraph's `interrupt()` — every path to `execute` goes through this same gate, none skip it. The paused run resumes with `Command(resume=True/False)`, using the same `thread_id` (requires a checkpointer, see `graph/build.py`).

`†` `execute` is the only node with side effects (`tools/github_client.py`'s `post_comment` / `add_label`). It posts `state["proposed_action"]` **verbatim** — the exact thing shown at the approval gate, never recomputed — and refuses to run at all without `state["approved"] is True`, checked in the node itself as a second, independent guarantee on top of the graph routing.

Core concepts

The interesting engineering in this project isn't any single LLM call — it's the handful of structural decisions that keep those calls from being trusted more than they've earned.

Reasoning and execution are different code paths, not different prompts

Every node up through `independent_review` only ever *proposes*. `execute` is the only function in the codebase allowed to call `tools/github_client.py`'s writes, and it isn't reachable except through `await_approval`'s gate — not by graph routing convention, but because `execute` itself refuses to run without `state["approved"] is True`, checked in the node body. If a future refactor added a shortcut edge straight to `execute`, this check would still catch it. The alternative — trusting a prompt to "always ask before acting" — fails exactly when it's needed most, since that's a behavior, not a boundary.

Independent review is a second call, not a second opinion from the same call

`independent_review` critiques `generate_diagnosis`'s output with a separate `ChatOpenAI` invocation and its own prompt — "independent" is enforced by *not sharing a reasoning trace*, not by asking the same model to "double check itself" in one call. More importantly, the actual approve/escalate decision (`groundedness_ok`, `risk_ok`, `permission_ok`) is three booleans **computed in Python**, not parsed from the LLM's response. The second model call only ever produces `reasoning` — a human-readable critique nobody's approval hinges on. This is the general shape for any "LLM-as-judge" pattern that has to be trustworthy: the judge explains, code decides.

RAG grounding is checked, not assumed

A citation being a real, retrieved snippet id used to be enough to count as "grounded" — but a real id can still point at a snippet that's barely related to the issue at hand, if that's simply the closest match a small corpus has to offer. `tools/retrieval.py` now returns each snippet's cosine similarity alongside its text; `independent_review.py` requires every cited id to clear `MIN_RELEVANCE_SCORE` (0.35, picked by directly measuring real query scores — see the module docstring for the actual numbers, not a guess) before counting as grounded. A genuinely novel issue with nothing similar in the corpus now correctly falls through to `escalate_to_human`, instead of a confident-sounding diagnosis getting rubber-stamped on noise. This is the retrieval-quality analogue of the same principle above: don't let "the model cited something" stand in for "the model cited something real."

A paused run is real state, not a client-side illusion

`await_approval` doesn't just show a preview and wait for a button click — it calls LangGraph's `interrupt()`, which actually **suspends the graph's execution mid-run**. Resuming requires the same `thread_id` and a real checkpointer (`AsyncPostgresSaver` in production; Render's free-tier disk is ephemeral, so a local-file checkpointer would silently lose every pending approval across a container recycle). "Approve" and "reject" on the frontend are `Command(resume=True/False)` calls into a graph that has been sitting paused in Postgres, possibly for hours, across completely separate HTTP requests. See docs/CHECKPOINTING.md for the mechanism in detail, with a sequence diagram.

Writes run as the person who approved them

The live app lets any GitHub user sign in and drive an analysis — reads and writes for that run use *their* OAuth token (`state["github_token"]`), not one shared credential belonging to the deployment owner. That's not just an auth nicety: it's what makes "the person approving this posted it" a statement GitHub itself can verify, rather than something ResolveFlow merely claims in its UI. It also means GitHub's own permission model does real work for free — a visitor analyzing someone else's repo can comment (open to any authenticated user) but can't necessarily add a label (needs triage/write access on that repo), and `execute.py` treats that as a partial success rather than crashing the whole request (see `CHANGELOG.md`).

Two different bars for two different kinds of eval

`eval/`'s suite (`uv run python -m eval.harness`) is a **regression** suite: `classify()`'s routing precedence, `independent_review()`'s gate, `execute()`'s permission check — each has one unambiguous correct answer, and the bar is 100% forever (`pass^k`: one bypassed gate across any number of trials is a critical failure, not something to average away). That's a different bar from a **capability** eval on `generate_diagnosis`'s actual diagnosis quality, which is model-graded, expected to improve over time, and never was going to sit at 100% — see the Roadmap. Conflating the two is a common mistake: a safety gate that "mostly" holds isn't a safety gate.

Status

Piece	State
`fetch_evidence` / `normalize_evidence`	✅ Real GitHub REST calls, validated into `IssueEvidence`
`classify`	✅ Rule-based routing (deliberate — see the node's docstring)
`generate_diagnosis`	✅ Real OpenAI call, structured `Diagnosis` output with citations, grounded via real Pinecone retrieval
`independent_review`	✅ Separate OpenAI critique call; approve/escalate gate computed in code, not trusted from the LLM
Retrieval corpus	✅ Real Pinecone index, ~2,750 chunks, weekly auto re-ingest, relevance-gated citations. ▶ details
`execute` + human approval gate	✅ Real `interrupt()` pause, resumed via `Command(resume=...)`, verified byte-for-byte end-to-end. ▶ details
React frontend + FastAPI backend	✅ Deployed (Vercel + Render), async backend, Postgres-checkpointed, live pipeline stepper. ▶ details
GitHub OAuth ("Sign in with GitHub")	✅ Any visitor signs in with their own account; writes run as them, bounded by a per-user daily cap. ▶ details
`eval/`	✅ Two-tier eval framework: ▶ safety gates + capability evals
Tests	❌ Unit/integration tests not written yet.

See `CHANGELOG.md` for the build history, and `PLAN.md` for the original 5-day/2-person plan this was compressed from into a solo, thinner end-to-end slice.

Setup

```
uv sync                 # installs from pyproject.toml
cp .env.example .env    # fill in GITHUB_TOKEN, OPENAI_API_KEY, PINECONE_API_KEY, DATABASE_URL
```

To let visitors sign in with their own GitHub account (rather than every request running under `GITHUB_TOKEN`), create a GitHub OAuth App at github.com/settings/developers with its callback URL set to `<your backend>/api/auth/github/callback`, then fill in `GITHUB_OAUTH_CLIENT_ID` / `GITHUB_OAUTH_CLIENT_SECRET` / `FRONTEND_URL` in `.env` (see `.env.example` for details, including the HTTPS-only caveat on the session cookie).

For the weekly re-ingest workflow to run, add `OPENAI_API_KEY` and `PINECONE_API_KEY` as repository secrets (Settings → Secrets and variables → Actions → New repository secret) — the same values already in your `.env`. `GITHUB_TOKEN` doesn't need to be added; Actions provides one automatically, sufficient for the read-only calls `ingest.py` makes.

Evaluation

ResolveFlow has **two kinds of evals**, each with its own bar and semantics:

Safety Gates (Regression Suite)

Bar: 100% forever. One failure = critical bug.

```
uv run python -m eval.harness
```

Tests the hard boundaries: `classify()`'s routing, `independent_review()`'s gates, `execute()`'s permission check. Each test has one unambiguous correct answer. Failures indicate the system weakened, not that it needs more training data.

Current: 12/12 passing

Capability Evals

Bar: Model-graded, expected to improve over time.

```
# Extract real test cases from your Pinecone corpus
python -m eval.extract_real_test_cases

# Run capability evals against those real issues
uv run python -m eval.capability_harness
```

Tests output quality on `generate_diagnosis`: Is the severity a valid enum? Do citations match retrieved IDs (no hallucinations)? Are retrieved snippets actually relevant?

Current: 6/6 passing on real issues from facebook/react and microsoft/terminal

Re-run `extract_real_test_cases` after each `ingest.py` run to test against the latest corpus.

Why two eval types? A safety gate that "mostly" works isn't a gate at all. The ability to diagnose issues well is a different bar — it *should* improve as the corpus grows and the prompt tunes, and averaging failures together obscures what's actually broken.

Running

```
uv run uvicorn app.main:app --reload                    # FastAPI backend, for the React frontend
cd frontend && npm install && npm run dev               # React frontend (localhost:5173)
uv run python ingest.py                                 # (re)populate the Pinecone index from live
GitHub issues
python -m eval.extract_real_test_cases                  # extract test cases from corpus
uv run python -m eval.harness                           # run safety-gate regression tests
uv run python -m eval.capability_harness                # run capability evals
uv run pytest                                            # once tests exist
```

Live: see the badges/links above. No local UI is needed to try it — the deployed frontend talks to the deployed backend directly.

Roadmap

1. ✅ **Capability eval suite for `generate_diagnosis`** — model-graded diagnosis quality against real issues from the Pinecone corpus. Currently tests severity enum validity and citation grounding (no hallucinations). Extensible for retrieval quality, recommendation grounding, and consistency.
2. **More capability eval types** — retrieve relevance, recommendation grounding, diagnosis consistency (same issue → same diagnosis).
3. **CI/CD integration** — run both safety-gate and capability suites on every commit; set thresholds for gating deploys.
4. **Live monitoring** — scheduled daily eval runs on production to catch regressions.
5. Tests for the node functions and the compiled graph.

Source: `CHECKPOINTING.md`

Checkpoint resume mechanics

How `app/main.py`'s checkpointer persists a paused `interrupt()` across two genuinely separate HTTP requests — and why that's the whole reason it's `AsyncPostgresSaver` and not `SqliteSaver`.

The problem this replaces

The prior checkpointer, `SqliteSaver`, wrote to a local file (`checkpoints.db`) on the backend's disk. On Render's free tier, that disk is **ephemeral** — every spin-down/spin-up cycle after idle time hands the app a fresh container, and local disk contents don't survive that recycle.

Concretely: a run pauses at `await_approval` waiting on a human; the free instance idles out before anyone clicks Approve; the next request boots a new container with an empty `checkpoints.db`. The paused thread's history is simply gone — `Command(resume=...)` has nothing to resume.

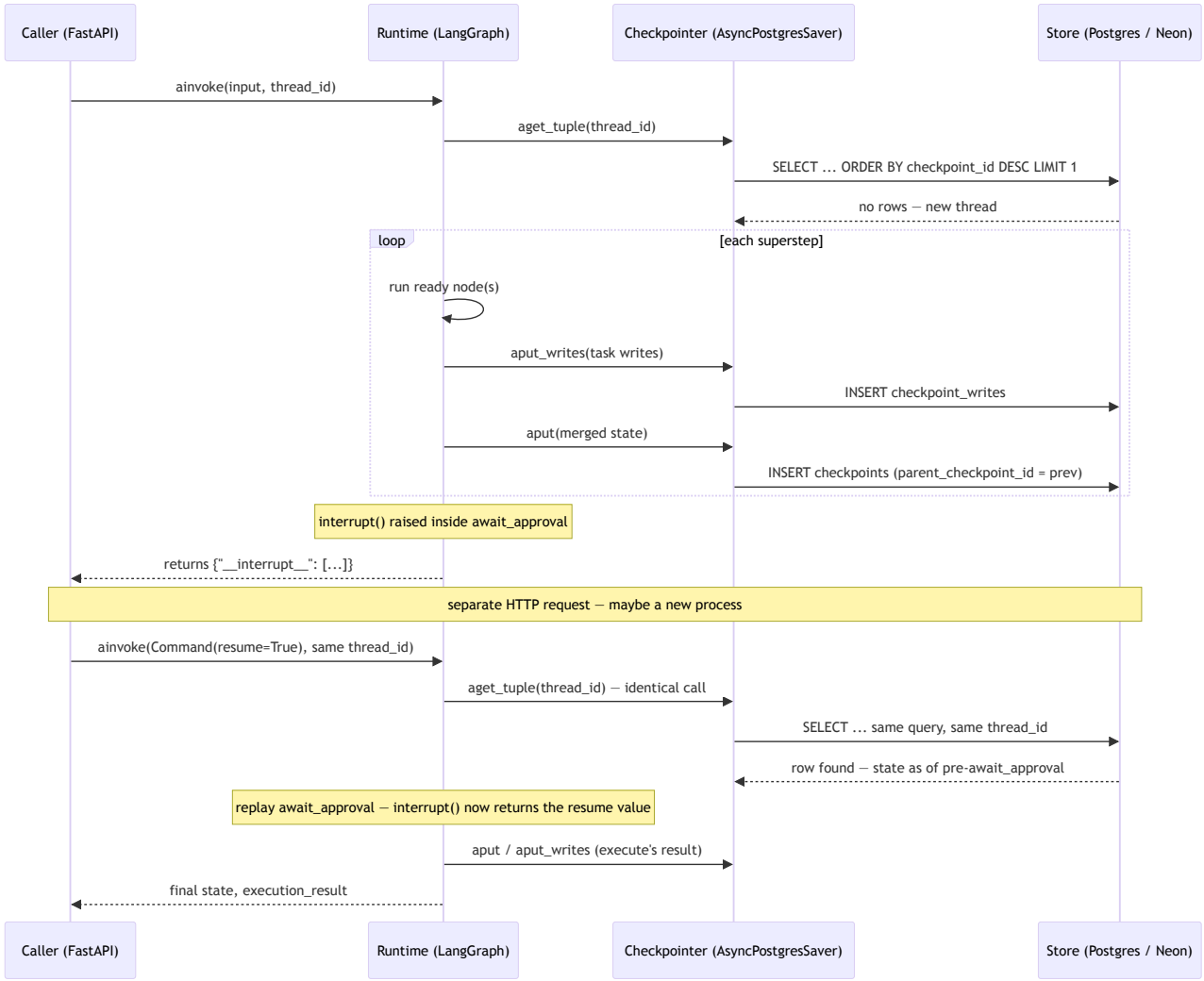
A checkpointer backed by a real database server isn't on the app host's disk at all. The container can be destroyed and rebuilt entirely — as long as `DATABASE_URL` still points somewhere real, every paused thread survives. That's the entire justification for the swap; nothing about `interrupt()`'s own logic changed.

The contract a checkpointer implements

LangGraph's runtime is written against one abstract interface — four async methods are all it needs, and it has no idea Postgres is underneath:

Method	Purpose
`aget_tuple()`	Fetch the latest checkpoint for a `thread_id` (or a specific one, given a `checkpoint_id`).
`aput()`	Persist a fully-merged state snapshot as a new checkpoint, linked to its parent.
`aput_writes()`	Stage one task's output before the whole superstep is merged — so a crash mid-superstep doesn't lose completed work.
`alist()`	Enumerate checkpoints for a thread — history/time-travel; not exercised by this app today.

The mechanism



The key thing this diagram makes explicit: **both `ainvoke()` calls run the exact same `aget_tuple` query.** There is no separate "resume" code path in the checkpointer. The only difference between "a fresh run" and "a resume" is whether a row already happens to exist for that `thread_id` when the query runs.

The schema behind it

Three tables, created by `saver.setup()` — confirmed by querying `pg_tables` directly against the live database after running it:

```
CREATE TABLE checkpoints (
    thread_id TEXT NOT NULL,
    checkpoint_ns TEXT NOT NULL DEFAULT '',
    checkpoint_id TEXT NOT NULL,
```

```
    parent_checkpoint_id TEXT,        -- links to the previous checkpoint
    checkpoint JSONB NOT NULL,        -- the full serialized GraphState
    metadata JSONB NOT NULL DEFAULT '{}',
    PRIMARY KEY (thread_id, checkpoint_ns, checkpoint_id)
);

CREATE TABLE checkpoint_writes (      -- one task's staged output,
    thread_id TEXT NOT NULL,          -- written before the whole
    checkpoint_ns TEXT NOT NULL,      -- superstep is merged
    checkpoint_id TEXT NOT NULL,
    task_id TEXT NOT NULL,
    idx INTEGER NOT NULL,
    channel TEXT NOT NULL,
    blob BYTEA NOT NULL,
    PRIMARY KEY (thread_id, checkpoint_ns, checkpoint_id, task_id, idx)
);

CREATE TABLE checkpoint_blobs (       -- large channel values, stored
    thread_id TEXT NOT NULL,          -- once and referenced rather than
    checkpoint_ns TEXT NOT NULL,      -- duplicated inline
    channel TEXT NOT NULL,
    version TEXT NOT NULL,
    blob BYTEA,
    PRIMARY KEY (thread_id, checkpoint_ns, channel, version)
);
```

Nothing is ever updated in place — every superstep inserts a **new** `checkpoints` row with `parent_checkpoint_id` pointing at the previous one. That's what makes a thread's full history genuinely replayable, not just its latest state.

One more fix that rides along: `graph/serde.py`

Every checkpoint round-trips `GraphState` through msgpack. Without telling the serializer which Pydantic classes are safe to (de)serialize, a future LangGraph version enforcing `LANGGRAPH_STRICT_MSGPACK` would refuse to store `IssueEvidence`, `Diagnosis`, or `ReviewResult` at all — this was previously a logged deprecation warning on every resume.

```
# graph/serde.py
from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer

_ALLOWED_MSGPACK_MODULES = [
    ("schemas.evidence", "IssueEvidence"),
    ("schemas.evidence", "CheckRun"),
    ("schemas.evidence", "LinkedPR"),
    ("schemas.diagnosis", "Diagnosis"),
    ("schemas.review", "ReviewResult"),
]

def get_serde() -> JsonPlusSerializer:
    return JsonPlusSerializer(allowed_msgpack_modules=_ALLOWED_MSGPACK_MODULES)
```

Passed into both checkpointers that exist in this codebase — `graph/build.py`'s in-process `MemorySaver`, and `app/main.py`'s `AsyncPostgresSaver` — so the fix applies regardless of which one is active.

What's actually been verified

- ✅ `saver.setup()` against the live Neon instance — 4 tables created (`checkpoints`, `checkpoint_writes`, `checkpoint_blobs`, `checkpoint_migrations`), confirmed via `pg_tables`.
- ✅ `app/main.py`'s real `lifespan` connects successfully on startup — not just an isolated test script.
- ✅ Two full real `analyze()` runs (real GitHub + OpenAI + Pinecone calls) completed and were checkpointed at each step.
- ❌ `interrupt()` → `Command(resume=...)` specifically — not yet exercised end-to-end against the real database. Both real issues tried during verification escalated to a human before ever reaching `await_approval` (correct, conservative behavior — `severity: high` always escalates by design — not a gap in this fix).

Source: `CAP_THEOREM_AND_AGENTIC_SYSTEMS.md`

CAP theorem — every angle an interviewer can pull on

---

1. The theorem itself — precise, not pop-sci

What CAP is actually a statement about: a replicated register under an asynchronous network. Not "databases" in general — a specific, narrow claim.

- **C (Consistency) = linearizability.** Every read returns the most recent completed write, as if there were one copy of the data and every operation happened atomically at a single instant. **This is not ACID's "C"** (which means invariants like foreign keys / constraints hold). Conflating the two is the single most common trip-up — if an interviewer asks "is CAP's C the same as ACID's C," the answer is an immediate, confident no, with the one-sentence reason above.
- **A (Availability) = every request to a non-failing node returns a non-error response in finite time.** Strict. "Eventually responds" doesn't count if it can block indefinitely. "The majority of nodes respond" doesn't count if even one healthy node errors out.
- **P (Partition tolerance) = the system keeps operating when the network arbitrarily drops or delays messages.**

The proof (be able to give this in ~30 seconds): Two nodes N1, N2 replicate `v=0`. Partition them completely. Client writes `v=1` to N1. A second client reads N2.

- N2 returns `0` → stale → not consistent.
- N2 blocks/errors until the partition heals → not available.
- No third option — N2 structurally cannot learn about the write while partitioned.

Why "pick 2 of 3" is wrong as a framing: partition tolerance isn't optional — networks partition regardless of what you want (cut cables, GC pauses, security group changes). So "CA" isn't a real category for a distributed system; it describes a single-node system, or one that will simply misbehave during a partition. **The only real choice is CP vs AP, and only during a partition.**

---

2. The gotcha questions that test whether you actually understand it (not just recite it)

"Doesn't Google Spanner give you both strong consistency AND high availability — doesn't that break CAP?" No. Spanner is **PC/EC** — it is *not* available during a partition (a minority partition can't reach quorum and refuses writes/reads). What Spanner actually did is minimize the *latency* cost of consistency in the non-partitioned case using TrueTime (GPS/atomic-clock-bounded clock uncertainty) to avoid a full coordination round-trip for read-

only transactions. It didn't repeal CAP; it optimized the "else" branch of PACELC. This is the #1 question that separates people who memorized "CAP says pick 2 of 3" from people who understand it.

"Is a single-node database CA?" Trivially, in a sense — no partitions are possible within one node, so the question doesn't apply. But this isn't a meaningful answer for distributed systems interviews; the moment you add a second node/replica, you're back to CP-or-AP-during-partition.

"What actually happens to the minority side of a partition in a CP system?" It can't form a write quorum (Raft/Paxos-style systems), so it refuses writes, and typically refuses reads too (to avoid serving stale data) — it becomes unavailable, not wrong. Compare to a system that keeps serving stale reads from the minority side while refusing writes — that's a hybrid, and you should be precise about which operations are affected.

"Give me a system that's CP and one that's AP, from memory."

- CP: etcd, ZooKeeper, Spanner, a single-leader RDBMS with **synchronous** replication.
- AP: Cassandra, DynamoDB (at a low consistency level) — every replica accepts reads/writes during a partition, conflicts reconciled later.

"How does an AP system reconcile conflicting writes after the partition heals?" Three mechanisms, know all three by name:

- **Last-write-wins (LWW)** — attach a timestamp/version, the higher one wins. Simple, but silently drops the loser's write — fine for a cache, dangerous for anything nobody's watching.
- **Vector clocks** — track causality per-replica so the system can detect *concurrent* (not causally ordered) writes and flag them for the application or the user to resolve, instead of silently picking one.
- **CRDTs (Conflict-free Replicated Data Types)** — data structures with a merge function that is provably commutative, associative, and idempotent, so *any* order of applying updates converges to the same final state with zero coordination. A grow-only counter, an OR-set, a LWW-register are canonical examples.

"Have you used anything CRDT-like yourself?" Yes — **LangGraph's reducers are exactly this shape.** `Annotated[list, operator.add]` is a commutative, associative merge function applied to concurrent writes from parallel branches in the same super-step, with no locking and no coordination between the writers. That's the same design principle as a CRDT, applied at the state-channel level instead of the database-replica level. Saying this unprompted is a strong signal — it shows you recognize the pattern outside its usual textbook context.

---

3. PACELC — the part that actually describes production systems

`if Partitioned: choose Availability or Consistency; Else: choose Latency or Consistency.`

The "else" branch is what CAP leaves out and what actually dominates your day-to-day: **even with a perfectly healthy network, strong consistency requires coordination** (a write reaching quorum, a read confirming the current leader), and that coordination costs a round trip. Cross-region, that's tens of milliseconds of physics you cannot optimize away.

Classifications to have cold:

- DynamoDB / Cassandra (default settings): **PA/EL** — available under partition, low-latency the rest of the time.
- Single-leader RDBMS, synchronous replication: **PC/EC**.
- Spanner: **PC/EC** — pays TrueTime commit-wait latency on every transaction to buy global strong consistency.

Probe: "Why would anyone choose PC/EC if it's slower?" Because for some data, a wrong answer is worse than a slow one — e.g., you'd rather a security agent's "has this host already been isolated" check block for 50ms than risk two replicas taking contradictory remediation actions on stale information.

---

4. Mapping the whole thing onto an agentic system (the part that shows lead-level judgment)

The wrong answer here is a single system-wide classification. The right answer is: **different components of the same agentic system deliberately sit on different sides of CP/AP, and naming which is which — and why — is the actual signal.**

Component	Classification	Why
Checkpoint / thread state store	**CP**	Two replicas resuming the same `thread_id` from divergent checkpoints can double-execute an already-approved action. The thread must be a linearizable object.
Retrieval / vector-index cache	**AP**	A stale competitive-intel paragraph costs almost nothing; a hard failure two minutes before a call costs a lot.
Idempotency / dedupe key store	**CP, but narrow**	Only the single atomic check-and-set on *one key* needs to be linearizable — not the whole system. This is the cheapest possible CP surface, and picking it out explicitly is the deep answer (§6 below).
Audit / compliance log	**CP**	An audit trail that can fork (two histories depending which replica you ask) isn't an audit trail — its entire job is being the undisputed record.
Multi-agent shared "hits"/scratch state (reducers)	**Effectively AP, resolved via CRDT-style merge**	Parallel branches write concurrently with no locking; the reducer (a commutative merge fn) resolves conflicts deterministically instead of blocking for coordination.

The line that lands in an interview: "*I don't pick one CAP answer for the whole agentic system — the checkpoint store has to be CP because a forked thread can double-execute a real-world action, the retrieval cache can be AP because staleness there is cheap, and the actual coordination cost gets pushed down to the smallest possible surface — one atomic key check — rather than making the whole system pay a consistency tax.*"

---

5. The unifying failure shape — recognize it across all three examples we hit today

Every one of these is **the same root cause wearing a different costume**: something acted on a read of state that hadn't yet caught up with a write that already happened, or already-included a value that then got double-counted.

1. **`interrupt()` replay** — code before the `interrupt()` call reruns on resume, because the node replays from the top; any side effect there fires twice.
2. **Subgraph reducer double-count** — a subgraph inherits the parent's already-accumulated value, and if the parent *also* uses an additive reducer at the boundary, it adds the whole (already-inclusive) returned value on top again.
3. **AP audit log, two replicas** — Replica B reads a stale view that doesn't yet show Replica A's write, concludes "not done," and redoes the action.

If an interviewer strings two of these together and asks "what's the common thread," this is the answer — not three unrelated bugs, one structural failure mode that shows up at every layer of a distributed/concurrent system: reducers, replayed execution, and eventual consistency.

---

6. Idempotency — the actual fix, in full depth

Making the *action* safe to repeat is cheaper than making the *whole system* strongly consistent. Delivery semantics (and CAP tradeoffs) stop mattering once double-delivery is harmless.

Step 1 — derive the key from intent, not delivery:

```
def idempotency_key(caller, tool, args) -> str:
    payload = json.dumps({"c": caller, "t": tool, "a": args}, sort_keys=True)
    return hashlib.sha256(payload.encode()).hexdigest()[:16]
```

`sort_keys=True` is load-bearing — `{"a":1,"b":2}` and `{"b":2,"a":1}` must hash identically or dedupe silently breaks on argument-order differences. Deriving from `(caller, tool, args)` rather than a message/delivery ID means it dedupes both redeliveries of the same message *and* two independently-triggered attempts at the same real-world action (exactly the two-replica scenario above).

Step 2 — the check-and-claim must itself be atomic, or you've just moved the race, not fixed it: Naive `if key not in store: store[key] = ...` is a check-then-act race — two concurrent callers can both pass the check before either writes. The fix is one atomic operation:

```
-- Postgres
INSERT INTO dedupe (key, result) VALUES ($1, $2)
ON CONFLICT (key) DO NOTHING
RETURNING key;   -- a row back = you won, go execute; nothing back = fetch the existing result
```

```
# Redis
if redis.set(key, result, nx=True):
    perform_action()
else:
    return redis.get(key)
```

Step 3 — store the result, not just the key. A duplicate caller must get back the *same answer* the original got, or it can't tell whether its own attempt succeeded.

Step 4 — TTL must outlive the maximum retry/resume horizon. If a paused agent run can resume days later (checkpointed), a 1-hour dedupe TTL reopens the exact hole you closed.

Step 5 — this atomic guarantee only holds within one store. If Replica A checks a primary and Replica B checks a stale read-replica of the "same" table, you're back to the original bug one layer down. The dedupe store itself must be the one genuinely-CP thing in the system — everything else can be AP around it.

The follow-up that catches people out: "what if `perform_action()` crashes *after* you've claimed the key but *before* it finishes?" This is the real production wrinkle. The naive two-step (claim key → execute) leaves a window where the key says "claimed" but the action never completed — and a retry sees "already claimed" and gives up, silently losing the action forever. The fix is a small state machine, not a boolean:

```
pending → in_progress → completed  (happy path)
                      → failed    (execution errored; safe to retry — the key transitions back to
pending, or a retry counter increments)
```

- Claim the key with state `pending` (atomic insert).
- Transition to `in_progress` right before executing (still atomic, still the single writer who claimed it).
- On success, write `completed` **plus the result**, in the same transaction/operation if possible.
- Add a **reconciliation sweep**: a background job that finds rows stuck in `in_progress` past some timeout and decides — did the action actually happen (check the external system's own state, e.g. "is host X actually isolated") or not — before deciding to retry or alert a human. This is the part most candidates skip: **you cannot always tell, from your own state alone, whether a crashed action completed** — sometimes you have to go ask the system of record itself (query CrowdStrike's own host-isolation status, not your local dedupe table) to resolve the ambiguity. That's the honest, senior answer, not "just retry."

---

7. Extreme depth — senior/staff-architect-level probes

These go past "do you know CAP" into "have you actually operated systems where this mattered." This is where a principal-level interviewer tries to find the edge of your knowledge.

Quorums — the tunable middle ground between CP and AP

Dynamo-style systems don't force a binary CP/AP choice — they expose **N** (replication factor), **W** (write quorum — how many replicas must ack a write), **R** (read quorum — how many replicas a read must consult).

The rule: `R + W > N` guarantees every read overlaps at least one replica that saw the latest write — i.e., read-your-writes / strong consistency, without needing all N replicas to be up. `R + W ≤ N` gives you higher availability and lower latency but reads can miss the latest write.

- `W=N, R=1`: fast reads, slow/fragile writes (all replicas must be up).
- `W=1, R=N`: fast writes, slow/fragile reads.
- `W=R=⌈(N+1)/2⌉` (majority quorum): the balanced default — tolerates `⌊N/2⌋` replica failures on either path.

Probe: "If R+W > N, is that system now CP?" No — it's *tunable-consistency AP*. A network partition can still isolate enough replicas that neither side can form its quorum, or both sides can (split quorum, rare but possible with bad config), so quorum systems are still fundamentally making an AP-style availability trade — they've just made the strength of consistency a per-request dial (`consistency_level=QUORUM` in Cassandra) instead of an architectural constant.

Read repair / hinted handoff / anti-entropy — how AP systems actually converge after the partition heals, without a human intervening:

- **Read repair**: on a read, if replicas disagree, the coordinator returns the latest value to the client *and* pushes the correction to the stale replicas in the background.
- **Hinted handoff**: if a replica is down at write time, another node stores a "hint" (the write, plus who it was really for) and replays it once the target comes back — bridges short outages without blocking the write.
- **Anti-entropy (Merkle trees)**: a background process periodically compares hash trees of each replica's data to find and fix divergence that read-repair never touched (cold data nobody's reading).

The full consistency spectrum — CAP's "C" is not binary

Interviewers testing depth will ask you to place systems on a spectrum, not just say "consistent or not":

- **Linearizability** (strongest, what CAP means) — real-time total order; once a write completes, every subsequent read anywhere sees it.
- **Sequential consistency** — all operations appear in *some* global order consistent with each process's own program order, but that order need not match real time.
- **Causal consistency** — operations that are causally related (a read that informed a later write) are seen in order everywhere; concurrent, unrelated operations can be seen in different orders on different replicas. This is usually the sweet spot for collaborative/multi-agent systems — cheaper than linearizability, still prevents "saw the reply before the question."
- **Eventual consistency** (weakest) — no ordering guarantee at all, only a promise that replicas converge *if writes stop*.

Session guarantees — practical middle-ground promises an AP system can still make to *one client*, without paying for global linearizability: read-your-own-writes, monotonic reads (never go backward in time on repeated reads), monotonic writes, writes-follow-reads. Concretely: an agent thread can be guaranteed to always see its *own* prior tool-call results (session consistency, cheap) even if the underlying store is globally AP and other threads might briefly see something stale.

FLP impossibility — the theorem CAP gets confused with

Fischer–Lynch–Paterson (1985): in a fully asynchronous system, no consensus protocol can guarantee both safety and termination (liveness) if even one node can fail. This is *not* CAP — CAP is about a replicated register's consistency/availability trade during partition; FLP is about whether distributed nodes can ever agree on a single value at all, in a network with no timing guarantees.

Why Raft/Paxos work in practice despite FLP: they don't defeat FLP — they sidestep it by assuming **partial synchrony** (messages eventually arrive within some bound, even if you don't know the bound in advance) and using randomized/staggered election timeouts to make the "two candidates split the vote forever" scenario vanishingly unlikely rather than impossible. **Probe: "So Raft can theoretically never terminate?"** Correct — in the worst adversarial case it can livelock forever; in practice randomized timeouts make that probability negligible. Know this distinction — confusing FLP and CAP in front of a principal engineer is an instant tell.

Consensus mechanics — what CP systems actually do under the hood

Both Paxos and Raft solve the same problem (get a cluster to agree on a sequence of values / a replicated log) via a leader + majority quorum:

- A leader is elected by majority vote (this is itself a consensus round).
- Every write goes through the leader, which replicates it to a majority before acknowledging — this **is** the R+W>N idea, specialized to "write quorum = majority, read = always from current leader."
- If the leader dies or is partitioned from the majority, a new election happens; the old leader, even if it comes back, cannot commit writes without rejoining the majority.

Split-brain and fencing tokens: what stops a partitioned old leader from still accepting writes and corrupting state? A **fencing token** — a monotonically increasing number issued at each leader election. Downstream storage rejects any write carrying an older token than the last one it accepted. So even if a stale leader doesn't know it's been deposed and keeps issuing writes, the storage layer — not the leader itself — is what actually enforces exclusivity. (Martin Kleppmann's canonical example: two "leaders" both holding what they think is a lock, both writing to shared storage — the storage's fencing-token check is the only thing that saves you, not the lock service's promise.)

CALM theorem — when can you skip coordination entirely?

Consistency As Logical Monotonicity: a program has a consistent, coordination-free distributed implementation **if and only if it is expressible as a monotonic computation** — one where new information never retracts old conclusions (only adds to them). Grow-only sets, max/min aggregation, and — directly relevant — **reducers like `operator.add` over independent contributions** are monotonic: order of arrival doesn't change the final answer, so no locking or coordination is needed to get a correct result. The moment a computation needs to *retract* something (delete, or "the most recent value wins" semantics like `operator.add` used the *wrong* way, per the subgraph double-count bug), it's non-monotonic and coordination becomes unavoidable. **This is the theoretical justification for why LangGraph's reducer model works without locks** — naming CALM by name is genuinely rare and reads as real distributed-systems depth, not interview cramming.

Byzantine faults — the failure model CAP doesn't cover

CAP and Paxos/Raft all assume **crash-stop or omission faults** — a node is either working correctly or silent/dead. They say nothing about a node that's up, responsive, and **lying** — sending different, self-serving answers to different peers. That's a **Byzantine fault**, and defending against it needs a different class of protocol (PBFT, or blockchain-style consensus), which typically needs **3f+1** nodes to tolerate f Byzantine actors, versus **2f+1** for simple crash-fault tolerance — because you need enough honest replies to outvote *any* coalition of liars, not just outnumber failures.

Why this is worth naming at a security company specifically: an insider threat, a compromised node, or a supply-chain-compromised dependency is a Byzantine actor by this model, not a crash fault — if a threat model includes "one of my own agent nodes could be compromised and start returning subtly wrong tool results," that's outside what CAP/Raft/Paxos protect against by design, and the honest answer is "that needs a different protocol family (or out-of-band attestation/signing), not a stronger consistency model."

Distributed transactions across agent tool calls — Sagas, not 2PC

An agent that calls three external tools in sequence (reserve budget → send email → update CRM) has the same shape as a distributed transaction across microservices. **Two-Phase Commit (2PC)** is the naive CP answer — a coordinator asks all participants to prepare, then commits only if all agree — but it's **blocking**: if the coordinator dies between prepare and commit, every participant sits holding a lock indefinitely. It also assumes every participant supports transactional prepare/commit, which a third-party API (Salesforce, Slack, a payment provider) generally doesn't.

The Saga pattern is the practical answer: each step commits immediately and independently; if a later step fails, you run **compensating actions** for the steps that already succeeded (refund the reservation, send a correction email) rather than rolling back a distributed transaction that was never atomic to begin with. This trades atomicity for availability — same fundamental trade as CAP, one layer up the stack — and it composes naturally with the idempotency work from §6: each step *and* each compensating action should itself be idempotent, because a saga coordinator retrying a failed step is exactly the same double-execution risk already covered.

The transactional outbox pattern — atomically writing state AND triggering a side effect

Classic problem: you need to **both** update your own DB *and* reliably notify something else (publish an event, trigger a downstream tool call) — but a DB write and a message-bus publish can't be one atomic transaction across two different systems. If you write to the DB first and then the publish fails, you've silently lost the notification; if you publish first and the DB write fails, you've notified about something that didn't happen.

Fix: write the DB change **and** the "event to publish" into the **same local database transaction** (the event goes into an `outbox` table, not directly onto the bus). A separate relay process reads unpublished rows from the outbox and publishes them, retrying as needed — and because the outbox write was transactional with the state change, they're always consistent with each other: you never have a state change with no corresponding event, or an event with no underlying change. The relay's publish-and-mark-sent step needs the same idempotent-consumer handling on the receiving end (§6) to be safe under its own retries.

---

8. Rapid-fire — drill these the morning of

Probe	One-line answer
CAP's C vs ACID's C	CAP = linearizability; ACID = invariants (FKs, constraints) hold. Not the same.
"Pick 2 of 3" — true?	No — partition tolerance isn't a choice; the real fork is CP vs AP, and only during a partition.
Does Spanner break CAP?	No — it's PC/EC; TrueTime cuts the latency cost of consistency, doesn't grant availability during a partition.
Name a CP system / an AP system	CP: etcd, ZooKeeper, Spanner, sync-replicated RDBMS. AP: Cassandra, DynamoDB (low consistency level).
PACELC's "else" branch, in one line	Even without a partition, consistency costs a coordination round trip = a latency tax, always.
How does AP reconcile conflicts	LWW (simple, can silently drop writes), vector clocks (detects concurrency, flags for resolution), CRDTs (merge function guarantees convergence with zero coordination).
A CRDT you've actually used	LangGraph reducers (`operator.add` etc.) — commutative/associative merges on concurrent super-step writes, no locking.

Probe	One-line answer
Why must the checkpoint store be CP	Two replicas resuming one `thread_id` from divergent checkpoints can double-execute an approved real-world action.
Why can the retrieval cache be AP	Staleness there is cheap (a slightly old paragraph); a hard failure is expensive.
What actually fixes double-execution	Idempotency at the action layer, not stronger consistency everywhere — shrink the CP surface to one atomic key check.
Why must the key check itself be atomic	Check-then-act is itself a race; two concurrent callers can both pass a non-atomic check before either writes.
What if execution crashes mid-claim	State machine (pending → in_progress → completed/failed) + a reconciliation sweep that queries the system of record, because your own state can't always tell you whether the action actually happened.
The one-sentence unifying insight	Every one of these bugs — interrupt() replay, subgraph reducer double-count, AP audit log, uncoordinated dedupe check — is the same root cause: acting on a read of state that hasn't caught up with a write that already happened.
R+W>N — what does it guarantee	Every read overlaps at least one replica holding the latest write — tunable consistency, still fundamentally an AP-style trade, not CP.
FLP vs CAP — the difference	FLP: can nodes ever agree on a value in an async network with failures (no, not with guaranteed termination). CAP: does a replicated register stay consistent AND available during a partition (no). Different theorems.
Why does Raft work despite FLP	It assumes partial synchrony + randomized election timeouts — sidesteps FLP's worst case rather than disproving it; a true worst-case livelock is still theoretically possible.
What stops a deposed leader from corrupting state	A fencing token (monotonically increasing) — storage rejects writes carrying a stale token; the storage layer enforces exclusivity, not the leader's own belief that it's still in charge.
CALM theorem, one line	A computation is coordination-free-consistent iff it's monotonic (never retracts conclusions) — why `operator.add`-style reducers need no locks.
Byzantine vs crash fault	Crash fault: a node is silent/dead. Byzantine fault: a node is up and actively lying. CAP/Raft/Paxos only defend against the former; a compromised node needs PBFT-style protocols (3f+1 nodes) instead.
2PC vs Saga for a multi-tool agent transaction	2PC is blocking and needs every participant to support prepare/commit (third-party APIs usually don't); Sagas commit each step immediately and use compensating actions on failure — trades atomicity for availability, same trade as CAP one layer up.
Transactional outbox pattern, one line	Write the state change and the "event to publish" in one local DB transaction (an outbox table); a relay publishes from it — guarantees the event and the state change are never inconsistent with each other.

Source: `DEPLOYMENT.md`

Deployment Guide

ResolveFlow is deployed on:

- **Frontend:** Vercel (React app)
- **Backend:** Render (FastAPI)
- **Database:** Postgres (managed)

Current URLs

- **Frontend:** https://resolveflow-web-officialbidishas-projects.vercel.app
- **API:** https://resolveflow-1h99.onrender.com/api
- **Health Check:** https://resolveflow-1h99.onrender.com/api/health

Auto-Deploy on Push

The `.github/workflows/deploy.yml` workflow automatically:

1. Runs safety-gate evals on every push to main
2. Deploys frontend to Vercel (if evals pass)
3. Deploys backend to Render (if evals pass)

Flow:

```
Push to main
    ↓
Run eval.harness (12/12 safety gates)
    ↓
Deploy frontend + backend (if passing)
    ↓
Live at URLs above
```

Setup Requirements

GitHub Secrets (for CI/CD)

Set these in GitHub Settings → Secrets and variables → Actions:

For Vercel:

- `VERCEL_TOKEN` — Personal access token (get from Vercel dashboard)
- `VERCEL_ORG_ID` — Your Vercel org ID
- `VERCEL_PROJECT_ID` — Your Vercel project ID

For Render:

- `RENDER_SERVICE_ID` — Your Render service ID
- `RENDER_API_KEY` — Your Render API key

For Evals:

- `OPENAI_API_KEY` — Your OpenAI key (already set)
- `PINECONE_API_KEY` — Your Pinecone key (already set)

Environment Variables

Vercel (Frontend)

```
VITE_API_URL=https://resolveflow-1h99.onrender.com
```

Set in Vercel Project Settings → Environment Variables

Render (Backend)

```
DATABASE_URL=postgresql://...    # Postgres connection string
OPENAI_API_KEY=sk-...            # OpenAI key
```

```
PINECONE_API_KEY=...             # Pinecone key
GITHUB_OAUTH_CLIENT_ID=...       # GitHub OAuth app ID
GITHUB_OAUTH_CLIENT_SECRET=...   # GitHub OAuth secret
FRONTEND_URL=https://resolveflow-web-officialbidishas-projects.vercel.app
```

Set in Render Dashboard → Environment

Manual Deployment

Deploy Frontend (Vercel)

```
# One-time setup
npm install -g vercel

# Deploy
vercel --prod

# Or use the Vercel dashboard to trigger deploys
```

Deploy Backend (Render)

```
# The render.yaml config handles everything
# Push to main → Render auto-deploys

# Manual trigger via Render dashboard:
# 1. Go to https://dashboard.render.com
# 2. Select your service
# 3. Click "Manual Deploy" → "Deploy latest commit"
```

Monitoring Deployments

Check Vercel Deployment

```
vercel logs --prod
# Or view in Vercel dashboard
```

Check Render Deployment

```
# View logs in Render dashboard
# Or check health:
curl https://resolveflow-1h99.onrender.com/api/health
```

Check Database

```
# Connect to Postgres
psql $DATABASE_URL

# Verify new tables exist (from feedback system)
```

```
\dt feedback
SELECT COUNT(*) FROM feedback;
```

Rollback

If a deployment breaks production:

Vercel

1. Go to Vercel dashboard → Deployments
2. Find the last working deployment
3. Click "Rollback" → Confirm

Render

1. Go to Render dashboard → your service → Events
2. Find the last successful deploy
3. Click the deployment ID → "Redeploy"

Pre-Deployment Checklist

Before pushing to main:

- [ ] Run local evals: `python -m eval.harness`
- [ ] Run capability evals: `python -m eval.capability_harness`
- [ ] Test frontend locally: `cd frontend && npm run dev`
- [ ] Test backend locally: `uv run uvicorn app.main:app --reload`
- [ ] Check for uncommitted changes: `git status`
- [ ] Pull latest: `git pull origin main`

Performance & Scaling

Render Free Tier

Current setup runs on Render's free tier with:

- **Limits:** 0.5 GB RAM, auto-stops after 15 min inactivity
- **Cold starts:** ~30s on first request after inactivity
- **Workaround:** The `keep-warm.yml` workflow pings the API every 15 min

Scaling Up

To move to production capacity:

1. **Upgrade Render Plan** - Go to Settings → Instance Type - Choose Starter ($7/month) or higher

2. **Database** - Currently Postgres (managed) - Upgrade to dedicated instance if needed

3. **Frontend Caching** - Vercel has CDN by default - Enable aggressive caching in `vercel.json` for faster loads

Database Migrations

When schema changes land (like the new `feedback` table):

1. **Local:** Migrations run on `app.setup()`
2. **Prod:** Migrations run on Render startup

To force a migration in production:

```
# Connect to production Postgres
psql $DATABASE_URL

# Run SQL manually if needed
CREATE TABLE IF NOT EXISTS feedback (
  id SERIAL PRIMARY KEY,
  ...
);
```

Troubleshooting

"Deploy failed" in GitHub Actions

```
# Check what failed
git log --oneline -5
# Was it eval failures or deploy failure?

# Run evals locally
python -m eval.harness

# If evals fail, fix the code before pushing again
```

Frontend showing "API unreachable"

```
# Check if backend is up
curl https://resolveflow-1h99.onrender.com/api/health

# If backend is down:
# 1. Check Render dashboard for errors
# 2. Verify DATABASE_URL is set in Render
# 3. Check Postgres connection
```

Slow cold starts

Backend is on free tier and gets cold-stopped after 15 min idle.

```
# The keep-warm workflow prevents this
# Check that .github/workflows/keep-warm.yml is running:
# Go to GitHub → Actions → "Keep Render warm"

# If still slow, upgrade to paid Render plan
```

Cost

Current estimated monthly cost:

- Vercel: $0 (free tier, up to 100 deployments/month)
- Render: $0 (free tier with cold-start penalty)
- Postgres: $0-50 (managed, depends on data size and connections)
- OpenAI: ~$5-10 (based on usage)
- Pinecone: $0 (free tier up to 1M vectors)

To reduce costs:

- Keep using free tiers
- Use scheduled evals instead of per-commit
- Batch inference jobs during off-peak hours

To scale up to production, budget ~$50-100/month total.

Source: `FEEDBACK_SYSTEM.md`

Customer Feedback System

ResolveFlow collects feedback from users on diagnosis quality and uses it to improve the eval suite and identify weak spots.

Architecture

```
User submits feedback
    ↓
/api/feedback/{thread_id} endpoint
    ↓
Stored in Postgres feedback table
    ↓
feedback_to_test_cases.py converts to eval cases
    ↓
capability_harness.py tests against them
    ↓
Regressions caught before deployment
```

API Endpoints

Submit Feedback

```
POST /api/feedback/{thread_id}
Authorization: Required (must be the thread owner)

Request body:
{
  "repo": "facebook/react",
  "issue_number": 12345,
  "verdict": "incorrect",  # "correct" | "incorrect" | "incomplete"
  "reason": "The diagnosis missed the root cause because..."
}

Response:
```

```
{
  "status": "feedback recorded"
}
```

Verdicts:

- `correct` — Diagnosis was accurate and actionable
- `incorrect` — Diagnosis was wrong or misleading
- `incomplete` — Diagnosis missed important context or alternatives

Get Feedback Stats

```
GET /api/feedback/stats
No auth required

Response:
{
  "correct": 42,
  "incorrect": 3,
  "incomplete": 7,
  "total": 52
}
```

Workflow

1. Customer Uses ResolveFlow

User analyzes an issue, sees the diagnosis, and provides feedback:

```
Issue analyzed → Diagnosis shown → User rates verdict + reason
```

2. Feedback Collected in Database

```
SELECT * FROM feedback WHERE verdict = 'incorrect';

id | thread_id | repo | issue_number | verdict | reason
---|-----------|------|--------------|---------|-------
1  | abc123... | my/repo | 42 | incorrect | "Missed that this was a race condition"
2  | def456... | my/repo | 43 | incomplete | "Didn't mention the performance impact"
```

3. Convert to Test Cases

When you want to regenerate eval test cases from customer feedback:

```
python -m eval.feedback_to_test_cases
```

This:

- Connects to Postgres
- Fetches all "incorrect" and "incomplete" feedback

- Generates `eval/customer_feedback_issues.py` with test cases
- Grouped by frequency (most-reported issues first)

4. Run Evals

Add customer feedback issues to your capability harness:

```
# eval/capability_harness.py
from eval.customer_feedback_issues import CUSTOMER_FEEDBACK_ISSUES

def run_customer_feedback_tests():
    """Test against issues where users reported problems."""
    results = []
    for evidence in CUSTOMER_FEEDBACK_ISSUES:
        test = DiagnosisSeverityValidTest(evidence=evidence)
        result = test.run()
        results.append(result)
    return results

def main():
    all_results = (
        run_diagnosis_severity_tests() +
        run_diagnosis_citations_tests() +
        run_customer_feedback_tests()  # ← ADD THIS
    )
```

5. Monitor & Improve

Track over time:

```
Week 1: 5 "incorrect" feedback → Add to evals → All tests fail → Fix root cause
Week 2: 2 "incorrect" feedback → Evals catch regression immediately
Week 3: 0 "incorrect" feedback → System improving!
```

Best Practices

For Users

- **Be specific** in the reason: "Missed the async context" beats "wrong"
- **Verify first**: Make sure the feedback is actually incorrect before submitting
- **One issue at a time**: Report separate problems as separate feedback

For the Team

- **Review feedback weekly**: Check the `/api/feedback/stats` dashboard
- **Re-run eval-to-test-cases monthly**: Capture accumulated customer reports
- **Add to evals quarterly**: Periodically merge customer feedback issues into the eval suite
- **Close the loop**: When a feedback-driven fix is deployed, celebrate it (and measure if the verdict rate improves)

Example Flow

Day 1: Customer Reports Problem

```
POST /api/feedback/thread-123
{
  "repo": "my-company/my-app",
  "issue_number": 567,
  "verdict": "incorrect",
  "reason": "The diagnosis said 'memory leak' but it was actually a deadlock in the async handler"
}
```

Day 3: Generate Test Cases

```
$ python -m eval.feedback_to_test_cases
Found 1 issues with negative feedback:
  my-company/my-app#567: incorrect (1 reports)
    → The diagnosis said 'memory leak' but it was actually a deadlock...

✓ Generated eval/customer_feedback_issues.py
```

Day 5: Run Evals Against Feedback

```
$ python -m eval.capability_harness
✗ FAIL | diagnosis_severity_valid | my-company/my-app#567
        Expected: low/medium/high, got: "critical"

✗ FAIL | diagnosis_citations_no_hallucination | my-company/my-app#567
        Invalid citations: ["doc-99"]
```

Day 7: Fix the System

Investigate why the diagnosis failed on this issue. Usually:

- The corpus doesn't have examples of this issue type → Add to ingest.py
- The prompt isn't specific enough → Tune generate_diagnosis prompt
- The retrieval is failing → Check MIN_RELEVANCE_SCORE in independent_review

Day 10: Verify Fix

Re-run feedback issues after deploying the fix — they should all pass now.

Metrics to Track

```
Dashboard should show:
- Feedback verdict distribution (% correct/incorrect/incomplete)
- Trending: Does % correct increase over time?
- Feedback volume: How many users are engaging?
- Issues with multiple reports: Which repos/patterns have problems?
```

Privacy & Security

- Feedback is only accessible to the user who submitted it (owned by `github_user_id`)
- The `/api/feedback/{thread_id}` endpoint requires auth and ownership check
- The `/api/feedback/stats` endpoint is public (aggregated stats only, no individual identities)
- No PII is stored; only repo/issue names and user's GitHub ID

Troubleshooting

No feedback showing up?

```
# Check if the feedback table has data
psql $DATABASE_URL -c "SELECT COUNT(*) FROM feedback;"

# See raw feedback
psql $DATABASE_URL -c "SELECT * FROM feedback LIMIT 10;"
```

Test cases not updating?

```
# Regenerate from scratch
python -m eval.feedback_to_test_cases

# Verify the file was created
cat eval/customer_feedback_issues.py
```

Stats endpoint showing zeros?

```
# Check database connectivity
python -c "from app.db import *; import asyncio; asyncio.run(get_feedback_summary(...))"
```

Source: `GITHUB_SECRETS_SETUP.md`

GitHub Secrets Setup for Auto-Deploy

ResolveFlow uses GitHub Actions to auto-deploy on every push to `main`. To enable auto-deployments, you need to set up GitHub Secrets.

Why Secrets?

The deploy workflow needs:

- **Vercel Token** — to deploy frontend
- **Render API Key** — to deploy backend
- **Service IDs** — to identify which services to deploy

These credentials should never be in the code, only in GitHub Secrets.

Setup Steps

1. Get Vercel Token

1. Go to https://vercel.com/account/tokens
2. Click "Create Token"
3. Name it `ResolveFlow-GitHub-Deploy`
4. Copy the token (it's long, starts with `v_...`)

2. Get Vercel Org/Project IDs

1. Go to your Vercel project: https://vercel.com/officialbidisha1/resolveflow-web-officialbidishas-projects
2. Open **Settings** → **General**
3. Find: - **Org ID** (under your name at top) - **Project ID** (under project name)
4. Copy both

3. Get Render API Key

1. Go to https://dashboard.render.com/account/api-tokens
2. Click "Create API Key"
3. Name it `GitHub-Deploy`
4. Copy the key

4. Get Render Service ID

1. Go to your Render dashboard: https://dashboard.render.com
2. Click your "resolveflow-api" service
3. Look at the URL or Settings → copy the Service ID - Format: `srv-1a2b3c4d5e6f7g8h`

5. Add to GitHub Secrets

1. Go to your GitHub repo: https://github.com/officialbidisha/ResolveFlow
2. Click **Settings** → **Secrets and variables** → **Actions**
3. Click **New repository secret** for each:

Secret Name	Value	From
`VERCEL_TOKEN`	Your Vercel token	Step 1
`VERCEL_ORG_ID`	Your Vercel org ID	Step 2
`VERCEL_PROJECT_ID`	Your Vercel project ID	Step 2
`RENDER_API_KEY`	Your Render API key	Step 3
`RENDER_SERVICE_ID`	Your Render service ID	Step 4

Note: The workflow checks if secrets exist. If missing, it logs a warning and skips that deploy step.

Verify Setup

After adding secrets:

1. Go to GitHub **Actions** tab
2. Push a test commit: `git commit --allow-empty -m "test: verify deployment"`
3. Watch the workflow run
4. Check that Vercel and Render deployments succeeded

Example: Step-by-Step

Get Vercel Token

```
visit: https://vercel.com/account/tokens
action: Create Token
copy: v_abc123def456...
```

Get Vercel IDs

```
visit: https://vercel.com/officialbidisha1/resolveflow-web
click: Settings → General
copy: Org ID = OrganizationId123
copy: Project ID = ProjectId456
```

Get Render Key

```
visit: https://dashboard.render.com/account/api-tokens
action: Create API Key
copy: rnd_abc123def456...
```

Get Render Service ID

```
visit: https://dashboard.render.com
click: resolveflow-api service
copy: Service ID = srv_1a2b3c4d5e6f7g8h
```

Add to GitHub

```
visit: https://github.com/officialbidisha/ResolveFlow/settings/secrets/actions
add: VERCEL_TOKEN = v_abc123def456...
add: VERCEL_ORG_ID = OrganizationId123
add: VERCEL_PROJECT_ID = ProjectId456
add: RENDER_API_KEY = rnd_abc123def456...
add: RENDER_SERVICE_ID = srv_1a2b3c4d5e6f7g8h
```

Test

```
git commit --allow-empty -m "test: verify deployment"
git push origin main
watch: https://github.com/officialbidisha/ResolveFlow/actions
```

Troubleshooting

"Secret not found" in logs

The workflow ran but a secret is missing. Add it following the steps above.

Vercel deploy fails with "permission denied"

Token might be:

- Expired — regenerate at https://vercel.com/account/tokens
- Wrong scope — create new token with "projects" scope
- Wrong account — make sure it's from the right Vercel account

Render deploy fails with "404"

- Service ID is wrong — copy exact ID from dashboard
- API key is expired — regenerate
- Service name changed — update RENDER_SERVICE_ID

Nothing deploys (secrets set, but no deploy happens)

1. Check GitHub Actions log: does it show "Skipping deploy"?
2. If yes, a required secret is missing — add it
3. If no, the evals (`eval.harness`) failed — fix those first

What Gets Deployed

Once secrets are set:

```
Push to main
    ↓
eval.harness runs (12/12 safety gates)
    ↓
If PASSING:
  ├─ vercel deploy → Frontend goes live
  └─ Render API call → Backend redeploys
    ↓
https://resolveflow-web-officialbidishas-projects.vercel.app (updated)
https://resolveflow-1h99.onrender.com/api (updated)
```

Security Notes

- Secrets are never printed in logs
- Each secret is masked as `***` in output
- Tokens should be long-lived but rotatable
- If a secret leaks, regenerate it immediately
- Don't commit credentials to the repo (GitHub prevents this anyway)

Need Help?

If deployment still fails after setting secrets:

1. Check the GitHub Actions log for the exact error
2. Verify each secret exists and is correct
3. Try manually deploying via Vercel/Render dashboards to isolate the issue
4. Check that `vercel.json` and `render.yaml` configs are valid

Still stuck? See `docs/DEPLOYMENT.md` for manual deployment steps.